

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	

10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.

7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.
9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Abstract

MEDI-EXPERT is a rule-based medical diagnosis expert system implemented in SWI-Prolog. The project converts a small, explicitly defined set of symptom-to-condition relationships into a knowledge base of facts and IF-THEN production rules. Given a patient's symptoms, the system can derive possible conditions using forward chaining and can test a proposed diagnosis using Prolog's goal-driven backward reasoning. The project also demonstrates declarative programming, unification, backtracking, reasoning traces, test scenarios, and an explanation facility.

The system is designed as an academic demonstration of expert-system principles rather than a clinically validated diagnostic tool. Its conclusions are limited to the rules encoded in the knowledge base and should not be used for real-world medical decision-making.

Keywords

Expert system, Prolog, SWI-Prolog, production rules, forward chaining, backward chaining, unification, backtracking, knowledge representation.

Assessment Alignment

The uploaded assessment requires an expert system developed in Prolog/SWI-Prolog and emphasizes production rules, forward chaining, backward chaining, declarative/non-procedural reasoning, unification, backtracking, testing, documentation, and demonstration. The report below is organized around the ten stages specified in the assessment.

Assessment Stage	Coverage in this Report
1. Domain Analysis	Sections 1–3
2. Knowledge Acquisition	Section 4
3. Knowledge Representation	Section 5
4. Inference Design	Sections 6–7
5. Paradigm Analysis	Section 8

6. System Development	Section 9
7. Testing	Section 10
8. Evaluation	Section 11
9. Documentation	Section 12
10. Demonstration	Section 13

Stage 1 – Domain Analysis

1.1 Problem Statement

Medical diagnosis involves relating observed symptoms to possible conditions. For an expert-system assignment, this reasoning can be represented as explicit logical rules. MEDI-EXPERT accepts a set of patient symptoms and determines which predefined diagnostic hypotheses are supported by those symptoms.

1.2 Objectives

Represent domain knowledge using Prolog facts and production rules.
Implement a rule-based reasoning process for symptom-to-condition inference.
Demonstrate forward chaining from known symptoms to conclusions.
Demonstrate backward chaining from a diagnostic goal to required symptoms.
Demonstrate unification and backtracking using Prolog queries.
Provide a simple explanation of why a conclusion was reached.
Test the system with positive, negative, and multi-result scenarios.
Document limitations and the scope of the knowledge base.

1.3 Domain Entities

Entity	Description	Examples
Patient	Person whose observations are analyzed	patient1, patient2
Symptom	Observable finding supplied to the system	fever, cough
Condition	Possible conclusion produced by a rule	common_cold, asthma
Rule	Logical relationship between symptoms and a condition	fever + cough → respiratory infection
Fact	Known information stored in the knowledge base	has_symptom(patient1, fever)
Diagnosis Goal	Condition that Prolog attempts to prove	diagnosis(patient1, asthma)

1.4 Input and Output

Component	Description
Input	Patient identifier and a set of symptoms.
Processing	Rule matching, unification, backtracking and inference.
Output	One or more possible conditions supported by the encoded rules.
Explanation	The rules/conditions responsible for a conclusion.

Stage 2 – Knowledge Acquisition

For this academic prototype, knowledge is represented as a deliberately small and transparent set of symptom combinations and possible conditions. The goal is to demonstrate knowledge acquisition and representation rather than reproduce a clinical diagnostic guideline.

2.1 Knowledge Categories

Respiratory observations: fever, cough, sore throat, wheezing, shortness of breath, chest tightness.
Upper-airway/allergy observations: runny nose, nasal congestion, sneezing, itchy eyes.
Systemic observations: fatigue, headache, body ache, nausea.

Gastrointestinal observations: abdominal pain, vomiting, diarrhea.

Urinary/metabolic observations: burning urination, frequent urination, increased thirst.

Other observations: skin rash, swollen lymph nodes, chest pain.

2.2 Knowledge Acquisition Method

Identify the target symptoms and possible conditions relevant to the educational scenario.

Express each relationship as an IF-THEN production rule.

Convert the antecedent conditions into Prolog predicates.

Check rules for duplicate or contradictory conclusions within the intended scope.

Create representative test cases that exercise both successful and unsuccessful inference.

2.3 Knowledge Scope

The rule base is intentionally limited. A real medical expert system would require validated clinical guidelines, expert review, broader differential diagnosis, patient history, medications, age/sex context, laboratory and imaging data where appropriate, uncertainty handling, and safety controls. Those capabilities are outside this academic prototype.

```
C:\Windows\System32\cmd.exe x + v
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- start.

=====
      MEDI-EXPERT
  Rule-Based Medical Expert System
=====
Academic demonstration only - not medical advice.

1. Show sample patients
2. Diagnose a sample patient
3. Forward chaining
4. Backward chaining
5. Explain a diagnosis
6. Test unification/backtracking
7. Run all demonstrations
8. Exit
Choose an option: 1
|: .

Sample patients loaded:
patient1: [fever,cough,fatigue,body_ache]
patient2: [sneezing,runny_nose,itchy_eyes]
patient3: [abdominal_pain,vomiting,diarrhea]
patient4: [wheezing,chest_tightness,shortness_of_breath]
patient5: [headache]

Invalid option. Please choose 0 to 7.
1. Show sample patients
2. Diagnose a sample patient
3. Forward chaining
4. Backward chaining
5. Explain a diagnosis
6. Test unification/backtracking
7. Run all demonstrations
8. Exit
Choose an option: |: 2
|: .
```

Stage 3 – Knowledge Representation

3.1 Facts

has_symptom(patient1, fever).

has_symptom(patient1, cough).

has_symptom(patient1, fatigue).

has_symptom(patient1, body_ache).

3.2 Predicates

Predicate	Meaning
has_symptom(Patient, Symptom)	Patient has the specified symptom.
diagnosis(Patient, Condition)	Condition can be derived for the patient.
rule(Condition, Conditions)	Forward-chaining rule definition.
symptom(Symptom)	Recognized symptom vocabulary.
explain(Patient, Condition)	Requests an explanation for a derived diagnosis.

3.3 Production Rules

Rule	IF conditions	THEN conclusion
R1	fever AND cough AND fatigue AND body_ache	influenza_like_illness
R2	fever AND cough AND sore_throat	respiratory_infection

R3	cough AND wheezing AND shortness_of_breath	asthma_related_condition
R4	sneezing AND runny_nose AND itchy_eyes	allergic_rhinitis
R5	fever AND cough AND shortness_of_breath	serious_respiratory_condition
R6	headache AND fever AND body_ache	viral_illness
R7	abdominal_pain AND vomiting AND diarrhea	gastrointestinal_infection
R8	diarrhea AND vomiting AND fatigue	dehydration_risk
R9	increased_thirst AND frequent_urination AND fatigue	diabetes_related_condition
R10	burning_urination AND frequent_urination AND abdominal_pain	urinary_tract_condition
R11	chest_pain AND shortness_of_breath	urgent_cardiac_or_respiratory_evaluation
R12	skin_rash AND fever AND fatigue	infectious_condition
R13	sore_throat AND swollen_lymph_nodes AND fever	throat_infection
R14	runny_nose AND sneezing AND nasal_congestion	common_cold
R15	cough AND runny_nose AND sore_throat	common_cold
R16	fever AND headache AND nausea	viral_condition
R17	fatigue AND increased_thirst AND frequent_urination	metabolic_condition
R18	abdominal_pain AND fever AND vomiting	abdominal_infection
R19	wheezing AND chest_tightness AND shortness_of_breath	asthma
R20	itchy_eyes AND sneezing AND runny_nose	allergy

Stage 4 – Inference Design

4.1 Forward Chaining

Forward chaining is data-driven. The engine begins with the patient's known symptoms, searches for rules whose conditions are satisfied, adds newly derived conclusions to the working fact set, and repeats until no additional rule can be fired.

Initial facts:

[fever, cough, fatigue, body_ache]

Matching rule:

fever + cough + fatigue + body_ache

↓

influenza_like_illness

Final derived conclusion:

influenza_like_illness

4.2 Forward-Chaining Algorithm

Read the initial fact set.
Find all production rules whose antecedents are satisfied.
Collect conclusions that are not already known.
If no new conclusion exists, stop.
Otherwise append the new conclusions to the fact set and repeat.

4.3 Backward Chaining

Backward chaining is goal-driven. The system starts from a proposed condition and tries to prove the conditions required by a matching rule. Prolog naturally performs this style of reasoning through its resolution mechanism, unification and backtracking.

```
Goal:
diagnosis(patient1, influenza_like_illness)
Required subgoals:
has_symptom(patient1, fever)
has_symptom(patient1, cough)
has_symptom(patient1, fatigue)
has_symptom(patient1, body_ache)
If all subgoals succeed:
true.
```

4.4 Comparison

Aspect	Forward Chaining	Backward Chaining
Direction	Facts → conclusions	Goal → required facts
Style	Data-driven	Goal-driven
Starting point	Known symptoms	Proposed diagnosis
Typical use	Derive all supported conclusions	Verify a specific hypothesis
Prolog relation	Implemented explicitly in the project	Naturally supported by Prolog rules

Stage 5 – Paradigm Analysis

5.1 Procedural vs Declarative Reasoning

Procedural Programming	Declarative / Non-Procedural Programming
Describes how to perform computation.	Describes facts and relationships.
Control flow is explicitly programmed.	The inference engine determines which rules apply.
Algorithm is central.	Knowledge and logical relationships are central.
Typical examples include imperative languages.	Prolog is a logic/declarative language.
Programmer controls many execution steps.	Programmer states what should hold and asks logical queries.

MEDI-EXPERT is declarative because the central knowledge is expressed as facts and logical rules. For example, the rule below states a relationship rather than specifying a step-by-step algorithm.

```
diagnosis(Patient, asthma) :-
has_symptom(Patient, wheezing),
has_symptom(Patient, chest_tightness),
has_symptom(Patient, shortness_of_breath).
```

Stage 6 – System Development

6.1 System Architecture

Module	Responsibility
Knowledge Base	Stores symptoms, patient facts and production rules.
Forward Inference Engine	Derives conclusions from known facts.
Backward Inference	Tests whether a proposed diagnosis can be proved.
Explanation Facility	Shows the rule/conditions supporting a conclusion.
User Interface Layer	Accepts patient information and displays results.

Testing Layer	Runs predefined scenarios and verifies expected results.
---------------	--

6.2 Core Prolog Implementation

```
diagnosis(Patient, influenza_like_illness) :-
has_symptom(Patient, fever),
has_symptom(Patient, cough),
has_symptom(Patient, fatigue),
has_symptom(Patient, body_ache).
rule(influenza_like_illness,
[fever, cough, fatigue, body_ache]).
all_true([], _).
all_true([H|T], Facts) :-
member(H, Facts),
all_true(T, Facts).
```

6.3 Forward-Chaining Engine

```
forward_chain(Facts, Conclusions) :-
forward_step(Facts, [], Conclusions).
forward_step(Facts, Conclusions, Conclusions) :-
findall(Diagnosis,
( rule(Diagnosis, Conditions),
all_true(Conditions, Facts),
\+ member(Diagnosis, Facts) ),
NewConclusions),
NewConclusions = [].
forward_step(Facts, OldConclusions, Conclusions) :-
findall(Diagnosis,
( rule(Diagnosis, Conditions),
all_true(Conditions, Facts),
\+ member(Diagnosis, Facts) ),
NewConclusions),
NewConclusions \= [],
append(Facts, NewConclusions, UpdatedFacts),
append(OldConclusions, NewConclusions, UpdatedConclusions),
forward_step(UpdatedFacts, UpdatedConclusions, Conclusions).
```

6.4 Explanation Facility

```
explain(Patient, Diagnosis) :-
diagnosis(Patient, Diagnosis),
write('Diagnosis: '),
write(Diagnosis), nl,
write('Reasoning: '), nl,
explain_rule(Patient, Diagnosis).
explain_rule(Patient, Diagnosis) :-
diagnosis(Patient, Diagnosis),
write(' Rule conditions satisfied for '),
write(Diagnosis), nl.
```

6.5 Execution Environment

Programming language: Prolog.

Recommended implementation environment: SWI-Prolog.

Main source file: medical_expert.pl.

Execution style: command-line Prolog queries and an optional interactive interface.

Stage 7 – Testing

The test plan includes positive cases, multiple-result cases, and a negative case. Each test is intended to verify that the corresponding production rules fire only when their required symptoms are present.

ID	Patient	Input symptoms	Expected result	Acceptance criterion
TC01	patient1	fever, cough, fatigue, body_ache	influenza_like_illness	Pass when R1 fires.
TC02	patient2	sneezing, runny_nose, itchy_eyes	allergic_rhinitis / allergy	Pass when R4 and/or R20 fires.

TC03	patient3	abdominal_pain, vomiting, diarrhea	gastrointestinal_infection	Pass when R7 fires.
TC04	new_patient	wheezing, chest_tightness, shortness_of_breath	asthma	Pass when R19 fires.
TC05	new_patient	headache only	No sufficient diagnosis	Pass when target diagnosis queries fail.
TC06	patient1	fever, cough, fatigue, body_ache	multiple supported respiratory/viral hypotheses may be returned	Pass when backtracking exposes alternatives.

7.1 Representative SWI-Prolog Queries

```
['medical_expert.pl'].
diagnosis(patient1, X).
diagnosis(patient1, influenza_like_illness).
test_forward.
test_backward.
explain(patient1, influenza_like_illness).
```

7.2 Negative Testing

A query such as `diagnosis(patient2, influenza_like_illness)` should fail if patient2 does not contain all antecedent symptoms required by the rule. This verifies that the engine does not assert a conclusion merely because one or two symptoms overlap.

Stage 8 – Evaluation

8.1 Evaluation Criteria

Criterion	Evaluation Approach
Correctness	Compare derived conclusions with expected rule outcomes.
Rule coverage	Use test cases that activate different sections of the knowledge base.
Inference consistency	Repeat equivalent queries and inspect deterministic rule satisfaction.
Forward reasoning	Verify facts are transformed into derived conclusions.
Backward reasoning	Verify a diagnosis can be proved only when its required subgoals succeed.
Unification	Use variable queries such as <code>diagnosis(patient1, X)</code> .
Backtracking	Request additional solutions using Prolog's alternative-solution mechanism.
Explainability	Display the supporting rule conditions for a derived diagnosis.

8.2 Strengths

Rules are explicit and easy to inspect.

The system is compact and suitable for demonstrating logic programming.

Forward and backward reasoning can be shown using the same knowledge base.

Prolog provides native unification and backtracking.

The reasoning process is easier to audit than an opaque predictive model.

8.3 Limitations

The knowledge base is intentionally small and educational.

The system does not model probabilistic uncertainty.

Symptoms are treated as symbolic facts rather than measured clinical observations.

It does not incorporate laboratory, imaging, medication, demographic or complete medical-history data.

The rules have not been presented as clinically validated diagnostic guidelines. Multiple matching rules can produce multiple possible conclusions; this is expected behavior in a rule-based system.

Stage 9 – Documentation

9.1 Recommended Repository Structure

```
medical-diagnosis-expert-system/  
├── README.md  
├── src/  
│   └── medical_expert.pl  
├── docs/  
│   ├── project_report.pdf  
│   ├── knowledge_base.md  
│   └── test_cases.md  
├── screenshots/  
│   ├── swipl_loaded.png  
│   ├── forward_chaining.png  
│   ├── backward_chaining.png  
│   └── test_results.png  
└── presentation/  
    └── medical_expert_system.pptx
```

9.2 README Contents

Project title and objective.
Software requirements and SWI-Prolog installation note.
Instructions to load medical_expert.pl.
Example queries and expected outputs.
Description of forward and backward chaining.
Testing summary.
Academic safety/scope disclaimer.
Repository structure and author details.

Stage 10 – Demonstration Plan

The demonstration should show the working program first and then connect the observed behavior to the expert-system concepts assessed in the rubric.

Open SWI-Prolog and load medical_expert.pl.
Show the knowledge base using listing(diagnosis). or listing(rule).
Run diagnosis(patient1, X). and explain that Prolog searches for matching conclusions.
Run diagnosis(patient1, influenza_like_illness). to demonstrate a successful goal.
Run diagnosis(patient2, influenza_like_illness). to demonstrate a failed goal.
Run test_forward. and explain the facts-to-conclusion sequence.
Use a query with a variable and press ; to demonstrate alternative solutions and backtracking.
Explain unification using the variable Patient or X in a diagnosis query.
Run explain(patient1, influenza_like_illness). and describe the explanation facility.
Show at least five test cases and conclude with limitations and future improvements.

10.1 Viva Questions and Short Answers

Question	Answer
What is an expert system?	A knowledge-based system that uses encoded domain knowledge and inference to derive conclusions.
Why Prolog?	Prolog directly supports facts, rules, logical queries, unification and backtracking, making it suitable for rule-based expert systems.
What is a production rule?	A conditional IF–THEN relationship represented as a rule in the knowledge base.
What is forward chaining?	A data-driven process that starts with known facts and repeatedly fires applicable rules.

What is backward chaining?	A goal-driven process that starts with a proposed conclusion and tries to prove its required conditions.
What is unification?	The matching process used by Prolog to make terms or variables compatible so that a rule can be applied.
What is backtracking?	Prolog's mechanism for returning to an earlier choice point and trying another possible solution.
Why can there be multiple diagnoses?	More than one production rule may be satisfied by the same set of symptoms.
What happens if no rule matches?	The requested diagnosis fails, or the system reports that no sufficient rule conditions were satisfied.
Is the system a real medical diagnostic tool?	No. It is an academic demonstration with a limited, non-clinically validated rule base.

Rubric Mapping and Expected Evidence

Rubric Area	Evidence in Project
Problem Analysis	Problem statement, objectives, domain entities, inputs/outputs.
Knowledge Acquisition & Representation	Knowledge categories, predicates, facts and 20 explicit production rules.
Production Rule / Knowledge Base Design	Structured IF-THEN rule table and Prolog rule representation.
Prolog Implementation	Knowledge base, diagnosis predicates, forward engine, explanation facility and test queries.
Forward Chaining	Explicit forward engine, algorithm, trace and test_forward query.
Backward Chaining	Prolog diagnosis rules, goal queries and backward reasoning trace.
Procedural vs Non-Procedural	Dedicated comparison and declarative rule examples.
Unification & Backtracking	Variable queries, alternative solutions and viva explanation.
Testing & Demonstration	Six representative test cases and step-by-step demo plan.
Documentation & GitHub	Repository structure, README requirements, screenshots and report organization.

Conclusion

MEDI-EXPERT demonstrates the complete lifecycle of a small rule-based expert system: domain analysis, knowledge acquisition, symbolic representation, inference design, Prolog implementation, testing, evaluation, documentation and demonstration. The project particularly highlights the strengths of Prolog for declarative knowledge representation and logical inference. Forward chaining is implemented as an explicit data-driven engine, while backward chaining is demonstrated through Prolog's natural goal-driven execution of rules. Unification and backtracking are made observable through variable queries and alternative solutions.

The project should be presented as an educational expert-system prototype. Its limited rule base and lack of clinical validation mean that it must not be interpreted as medical advice or as a substitute for a qualified healthcare professional.

Future Enhancements

Add a richer, expert-reviewed knowledge base.

Introduce certainty factors or probabilistic reasoning.

Add patient history and demographic context.
Provide a graphical user interface.
Store and visualize reasoning traces.
Add automated regression testing for every production rule.
Add conflict-resolution strategies when multiple rules fire.
Separate urgent-symptom alerts from ordinary diagnostic hypotheses.
Add multilingual symptom input for demonstration purposes.

References

1. SIMATS Engineering, Assessment Tool 1: Expert System Development Project, uploaded course assessment document. The report follows the ten-stage structure and rubric requirements specified in that document.
2. SWI-Prolog Documentation, official SWI-Prolog project documentation. Consult the official documentation for installation, Prolog syntax, queries, and execution.

Appendix A – Suggested Full Source File

The following is the core source structure used for the project. The final submitted .pl file should contain the complete rule set, sample patient facts, forward-chaining engine, backward-diagnosis predicates, explanation facility and testing predicates.

```
% MEDI-EXPERT - Core Prolog structure
:- dynamic has_symptom/2.
symptom(fever).
symptom(cough).
symptom(sore_throat).
symptom(fatigue).
symptom(body_ache).
symptom(headache).
symptom(runny_nose).
symptom(nasal_congestion).
symptom(sneezing).
symptom(itchy_eyes).
symptom(wheezing).
symptom(shortness_of_breath).
symptom(chest_pain).
symptom(chest_tightness).
symptom(abdominal_pain).
symptom(vomiting).
symptom(diarrhea).
symptom(increased_thirst).
symptom(frequent_urination).
symptom(burning_urination).
symptom(skin_rash).
symptom(swollen_lymph_nodes).
symptom(nausea).
diagnosis(Patient, influenza_like_illness) :-
has_symptom(Patient, fever),
has_symptom(Patient, cough),
has_symptom(Patient, fatigue),
has_symptom(Patient, body_ache).
diagnosis(Patient, respiratory_infection) :-
has_symptom(Patient, fever),
has_symptom(Patient, cough),
has_symptom(Patient, sore_throat).
diagnosis(Patient, asthma_related_condition) :-
has_symptom(Patient, cough),
has_symptom(Patient, wheezing),
has_symptom(Patient, shortness_of_breath).
diagnosis(Patient, allergic_rhinitis) :-
has_symptom(Patient, sneezing),
has_symptom(Patient, runny_nose),
has_symptom(Patient, itchy_eyes).
diagnosis(Patient, serious_respiratory_condition) :-
has_symptom(Patient, fever),
has_symptom(Patient, cough),
has_symptom(Patient, shortness_of_breath).
diagnosis(Patient, gastrointestinal_infection) :-
```

```

has_symptom(Patient, abdominal_pain),
has_symptom(Patient, vomiting),
has_symptom(Patient, diarrhea).
diagnosis(Patient, common_cold) :-
has_symptom(Patient, runny_nose),
has_symptom(Patient, sneezing),
has_symptom(Patient, nasal_congestion).
diagnosis(Patient, common_cold) :-
has_symptom(Patient, cough),
has_symptom(Patient, runny_nose),
has_symptom(Patient, sore_throat).
diagnosis(Patient, asthma) :-
has_symptom(Patient, wheezing),
has_symptom(Patient, chest_tightness),
has_symptom(Patient, shortness_of_breath).
diagnosis(Patient, allergy) :-
has_symptom(Patient, itchy_eyes),
has_symptom(Patient, sneezing),
has_symptom(Patient, runny_nose).
rule(influenza_like_illness, [fever,cough,fatigue,body_ache]).
rule(respiratory_infection, [fever,cough,sore_throat]).
rule(asthma_related_condition, [cough,wheezing,shortness_of_breath]).
rule(allergic_rhinitis, [sneezing,runny_nose,itchy_eyes]).
rule(serious_respiratory_condition, [fever,cough,shortness_of_breath]).
rule(gastrointestinal_infection, [abdominal_pain,vomiting,diarrhea]).
rule(common_cold, [runny_nose,sneezing,nasal_congestion]).
rule(common_cold, [cough,runny_nose,sore_throat]).
rule(asthma, [wheezing,chest_tightness,shortness_of_breath]).
rule(allergy, [itchy_eyes,sneezing,runny_nose]).
all_true([], _).
all_true([H|T], Facts) :-
member(H, Facts),
all_true(T, Facts).
forward_chain(Facts, Conclusions) :-
forward_step(Facts, [], Conclusions).
forward_step(Facts, Conclusions, Conclusions) :-
findall(Diagnosis,
(rule(Diagnosis, Conditions),
all_true(Conditions, Facts),
\+ member(Diagnosis, Facts)),
NewConclusions),
NewConclusions = [].
forward_step(Facts, Old, Conclusions) :-
findall(Diagnosis,
(rule(Diagnosis, Conditions),
all_true(Conditions, Facts),
\+ member(Diagnosis, Facts)),
NewConclusions),
NewConclusions \= [],
append(Facts, NewConclusions, UpdatedFacts),
append(Old, NewConclusions, Updated),
forward_step(UpdatedFacts, Updated, Conclusions).
backward_diagnose(Patient) :-
findall(Diagnosis, diagnosis(Patient, Diagnosis), Diagnoses),
write('Possible conclusions: '), write(Diagnoses), nl.
explain(Patient, Diagnosis) :-
diagnosis(Patient, Diagnosis),
write('Diagnosis: '), write(Diagnosis), nl,
write('Rule conditions satisfied.'), nl.

```

Appendix B – Demonstration Commands

```

['medical_expert.pl'].
listing(diagnosis).
listing(rule).
diagnosis(patient1, X).
diagnosis(patient1, influenza_like_illness).
diagnosis(patient2, influenza_like_illness).
test_forward.
test_backward.
explain(patient1, influenza_like_illness).

```

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference	7–8% – Provides relevant test cases with mostly	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation , GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements .	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				